Todo App — Web Development Learning Project

Complete Context, Architecture & Progress Handoff

Purpose. Preserve the project's current architecture, decisions, progress, and next steps so this context can be fed into a future project/source without restarting completed work.

Project intent from the uploaded TODO. Learn an industry-style Node.js/MERN setup through a Todo app, with the longer-term intention of replacing MongoDB with MySQL and learning React through the same project. ■filecite■turn10file0■L1-L1■

1. Core Project Principles

- Use the Todo app as the learning project for real-world web development concepts.
- Keep the backend layered and database-agnostic enough that MongoDB can later be replaced by MySQL.
- Learn React through the actual application rather than isolated examples.
- Build for scalability, maintainability, reusability, and easy future modification.
- Use Tailwind CSS for the frontend.
- Centralize theme/design tokens so a global theme can be changed without editing many components.
- Avoid unnecessary abstraction and unnecessary dependencies.
- Do not endlessly harden the backend at the expense of getting the frontend built.

2. Backend — Current Architecture & Progress

The backend is substantially complete. It uses separation of concerns across models, repositories, services, controllers, middleware, configuration, and server startup.

```
server/ ■■■ src/ ■ ■■■ controllers/ ■ ■■■ services/ ■ ■■■ repositories/ ■ ■■■ models/ ■ ■■■ middleware/
■ ■■■ config/ ■ ■■■ app.js ■ ■■■ server.js ■■■ .env ■■■ package.json
```

Implemented backend capabilities

- **MongoDB/Mongoose:** dedicated connection function; server starts after DB connection succeeds.
- **Server lifecycle:** SIGINT graceful shutdown closes MongoDB and then shuts down the server.
- **Authentication:** registration and login.
- **Password security:** bcrypt hashing; password field is select:false in User model.
- **JWT:** access token with JWT_SECRET and 15-minute expiry.
- **Refresh tokens:** random token secret + tokenId; only bcrypt hash of the secret is stored.
- **Refresh-token service/repository:** create, verify, delete.
- **TTL:** refresh tokens use expiresAt with MongoDB TTL index.
- **Authorization:** authentication checks, role-based authorization, and Todo ownership checks.
- **Todo CRUD:** create, list by user, get by id/user, update by id/user, delete by id/user.
- **Validation:** Mongoose validation for Todo title, completed, priority, and required user.
- **Error middleware:** ValidationError, CastError, custom email duplicate, invalid credentials, JWT errors, authentication/authorization errors, refresh-token errors, user-not-found, oversized body, and generic 500.
- **Security middleware:** CORS, Helmet, rate limiting, exposed rate-limit headers, and request body-size limit.
- **Logging:** request logger logs method and URL.
- **Indexes:** User.email unique, RefreshToken.tokenId unique, RefreshToken.expiresAt TTL, Todo.user index.
- **E11000:** database-level duplicate-key fallback was added so unique-index race conditions can return 409.

Important backend decisions

- Todo queries include userId, preventing cross-user access even when another Todo's ObjectId is known.
- The normal duplicate-email check remains useful for UX; E11000 handling is the database-level fallback.
- Refresh-token revocation records are currently deleted rather than kept as a permanent revoked-token history; this was discussed and accepted for the current design.
- Backend should now be treated as effectively frozen while the frontend is built.

3. Backend — Remaining Later Work

- Final cleanup/refactor after frontend integration.
- Final Postman regression test and API endpoint documentation.
- Deployment/environment configuration.
- Optional later improvement: secure HttpOnly-cookie delivery for refresh tokens.
- Only add additional security features when they solve a real requirement.

4. Frontend — Current Progress

- **React + Vite:** created from scratch in the previously empty client directory and running successfully on localhost.
- **Project naming:** frontend package is intended as todo-frontend; backend package is todo-backend.
- **Root package cleanup:** client and server retain their own package.json/package-lock.json/node_modules; unnecessary root-level Node package artifacts were identified and removed.
- **React Router:** installed correctly inside client after an initial mistake of installing from the root.
- **Router architecture:** BrowserRouter in main.jsx; Routes/Route definitions in App.jsx.
- **Pages:** Home, Login, Register, Todos created and routes verified.
- **Tailwind CSS:** installed and verified successfully using Tailwind's current Vite integration.
- **Folders:** components, pages, services, context, hooks, utils created.
- **Theme system:** CSS variables + data-theme selectors + Tailwind semantic token mapping implemented and manually tested with default/light, dark, and purple themes.
- **ThemeContext.jsx:** created with theme state, localStorage persistence, document data-theme synchronization, and useTheme custom hook.

5. Frontend Target Structure

```
client/ ■■■ src/ ■■■ components/ ■■■ pages/ ■ ■■■ Home.jsx ■ ■■■ Login.jsx ■ ■■■ Register.jsx ■ ■■■ Todos.jsx ■■■ services/ ■■■ context/ ■ ■■■ ThemeContext.jsx ■■■ hooks/ ■■■ utils/ ■■■ App.jsx ■■■ main.jsx ■■■ index.css
```

6. Frontend Architecture Rules

- Tailwind CSS is the styling system.
- Use reusable components, but do not over-componentize tiny pieces.
- Pages compose screens; components provide reusable UI.
- API calls belong in a services layer, not scattered through components.
- Authentication state will be centralized in AuthContext/useAuth.
- Theme state is centralized in ThemeContext/useTheme.
- Avoid hard-coded hex colors inside components.
- Use semantic tokens such as primary, primary-hover, background, surface, text, text-muted, border, danger, success, and warning.
- Environment-specific API URLs/configuration should be centralized.
- Design changes should be possible from one central place wherever practical.

7. Theme System — What Has Been Built

Components use semantic utilities such as **bg-primary**, **bg-background**, **bg-surface**, **text-text**, **text-text-muted**, and **border-border**. These map through Tailwind's theme layer to CSS variables. The actual variables are overridden by the active data-theme attribute.

```
Component ↓ Tailwind semantic utility ↓ @theme mapping ↓ CSS variable ↓ [data-theme] override ↓ Actual visual color
```

Theme meanings:

- **primary:** main brand/action color — buttons, links, active states.
- **primary-hover:** hover state for primary actions.
- **background:** main page background.
- **surface:** cards, navbar, panels, forms — elements sitting above the background.
- **text:** main/important text.

- **text-muted:** secondary descriptions and metadata.
- **border:** input/card/separator borders.
- **danger/success/warning:** semantic states for destructive, successful, or warning UI.

8. ThemeContext — Current Design

ThemeContext uses createContext, useContext, useEffect, and useState. It initializes from localStorage or defaults to light. When theme changes it sets the html data-theme attribute and persists the value.

```
const ThemeContext = createContext(null); function ThemeProvider({ children }) { const [theme, setTheme] = useState(() => { return localStorage.getItem("theme") || "light"; }); useEffect(() => { document.documentElement.setAttribute("data-theme", theme); localStorage.setItem("theme", theme); }, [theme]); return ( {children} ); } function useTheme() { return useContext(ThemeContext); }
```

useTheme() is only a custom hook/shortcut for accessing the ThemeContext. It does not store the theme itself.

9. Current Immediate Next Tasks

- **1.** Confirm/wire ThemeProvider around BrowserRouter/App in main.jsx.
- **2.** Build a reusable ThemeSwitcher using useTheme(), with light/dark/purple options.
- **3.** Build the Navbar using React Router NavLink so active routes can be styled.
- **4.** Build the API service layer; install Axios only when the API layer is started.
- **5.** Build Register/Login UI.
- **6.** Build AuthContext/useAuth and protected routes.
- **7.** Connect authentication to the existing backend.
- **8.** Build Todo dashboard and CRUD UI.
- **9.** Add loading, error, empty, validation, responsive, and polished UX states.
- **10.** Final production build, environment setup, deployment, and backend regression.

10. Progress Snapshot

Area	Status	Current state
Backend architecture	DONE	Layered Node/Express/Mongoose
Authentication	DONE	Register, login, JWT, refresh tokens
Authorization	DONE	Auth, role, ownership
Todo CRUD	DONE	User-scoped CRUD
Security	DONE	CORS, Helmet, rate limit, body limit, errors
DB indexes	DONE	Unique + TTL + Todo.user
Graceful shutdown	DONE	SIGINT + DB close
React + Vite	DONE	Running
React Router	DONE	Verified
Tailwind	DONE	Verified
Frontend folders	DONE	Scalable base
Theme tokens	DONE	Centralized
Multiple themes	DONE	Light/dark/purple tested
ThemeContext	DONE*	Created; Provider wiring is next check
Navbar	NEXT	Use NavLink
Auth UI	PENDING	Not started
API integration	PENDING	Not started
Todo UI	PENDING	Not started
Deployment	PENDING	Later

11. Handoff Prompt for Future Project Context

```
Continue the Todo App Web Development Learning Project from its current state. Do NOT restart completed setup. Goal: build an industry-style scalable Todo app using Node.js/Express/MongoDB on the backend and React + Vite + Tailwind CSS on the frontend. Keep separation of concerns and make the DB layer replaceable later so MongoDB can be swapped for MySQL. Backend is substantially complete: layered controllers/services/repositories/models, registration/login, bcrypt, JWT access tokens, refresh-token hashing and TTL, authentication, authorization, ownership checks, Todo CRUD, validation, centralized error middleware, CORS, Helmet, rate limiting, body-size limits, logger, graceful shutdown, database indexes, and E11000 duplicate-key handling. Backend is frozen for now except meaningful integration/production polish. Frontend is currently running. React Router is configured with BrowserRouter in main.jsx and Routes in App.jsx. Home/Login/Register/Todos pages exist and routing is verified. Tailwind is working. Scalable folders exist: components, pages, services, context, hooks, utils. A centralized multi-theme system is working using CSS variables, data-theme, and Tailwind semantic tokens. Light/default, dark, and purple themes were manually tested. ThemeContext.jsx exists with localStorage persistence and useTheme. Frontend design rules: use Tailwind; reusable components without over-abstraction; API calls in services; AuthContext/useAuth for auth; ThemeContext/useTheme for themes; semantic design tokens instead of hard-coded colors; centralized environment configuration; easy global modification. Next task: wire/verify ThemeProvider in main.jsx, then create ThemeSwitcher, then Navbar with NavLink, then API service layer, authentication UI/context/protected routes, Todo UI, UX polish, and finally deployment.
```